

Experiment No: 02

Date: 17/09/2026

Title: Library Management System using Object-Oriented Programming Concepts

Objective / Aim

To design and implement a Library Management System in Java using object-oriented programming concepts such as encapsulation, constructors, inheritance, method overloading, method overriding, abstraction, and interfaces.

System Requirements

Hardware

- Standard PC/Laptop
- Intel Core i3/i5 processor or equivalent
- Minimum 4 GB RAM
- Minimum 500 MB free storage

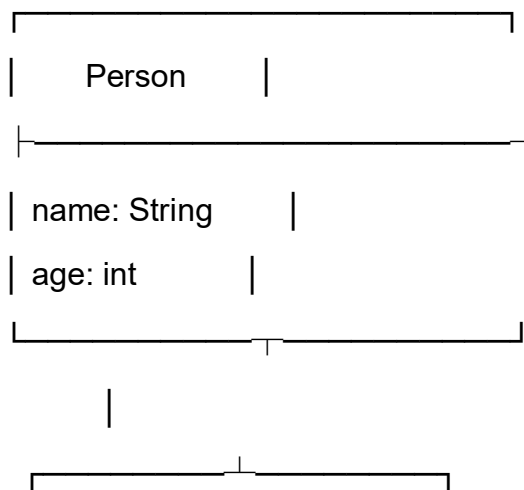
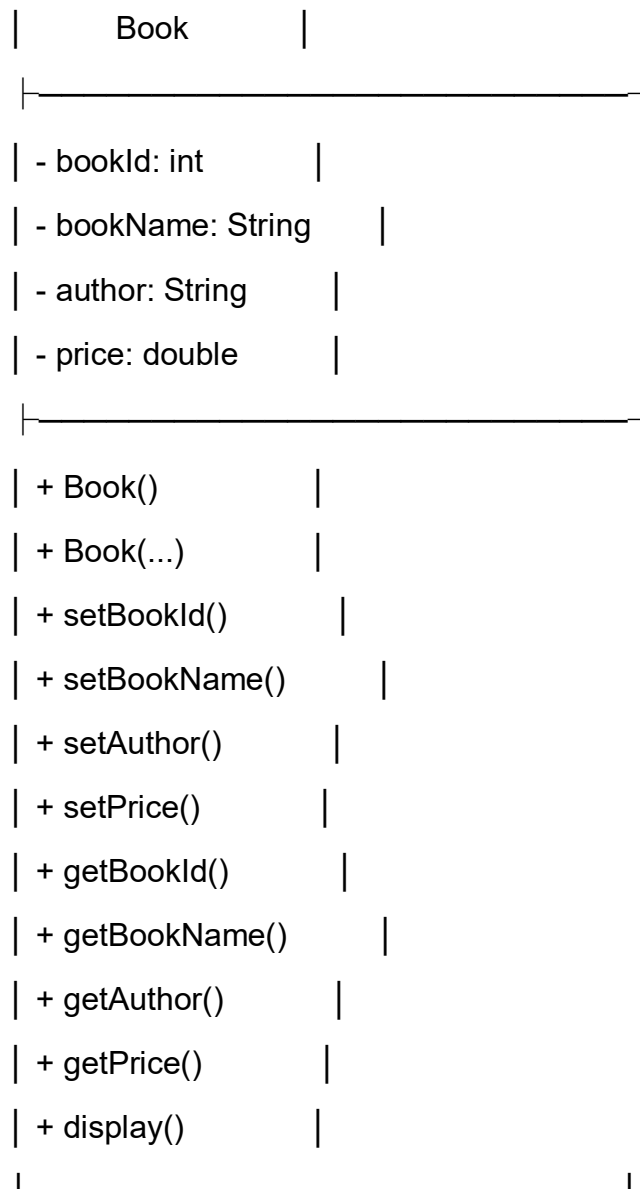
Software

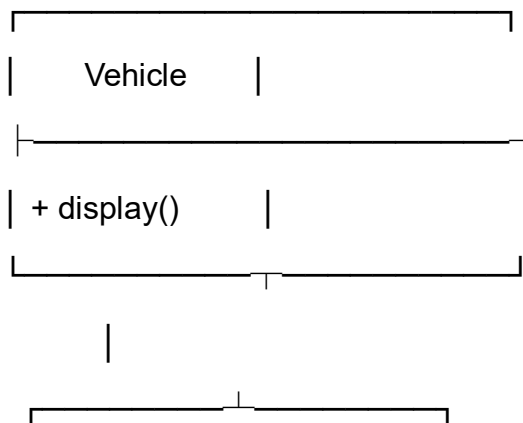
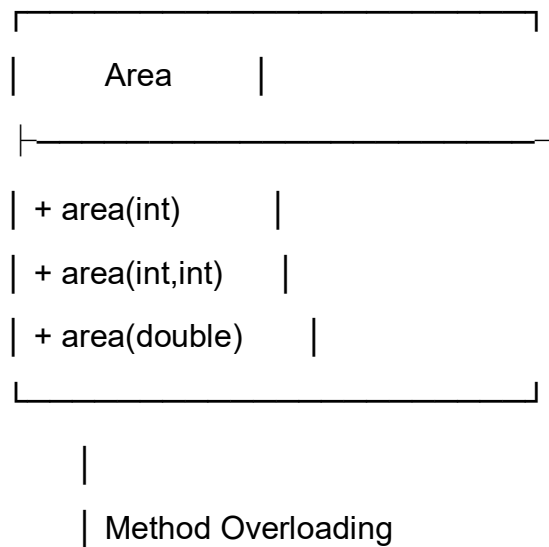
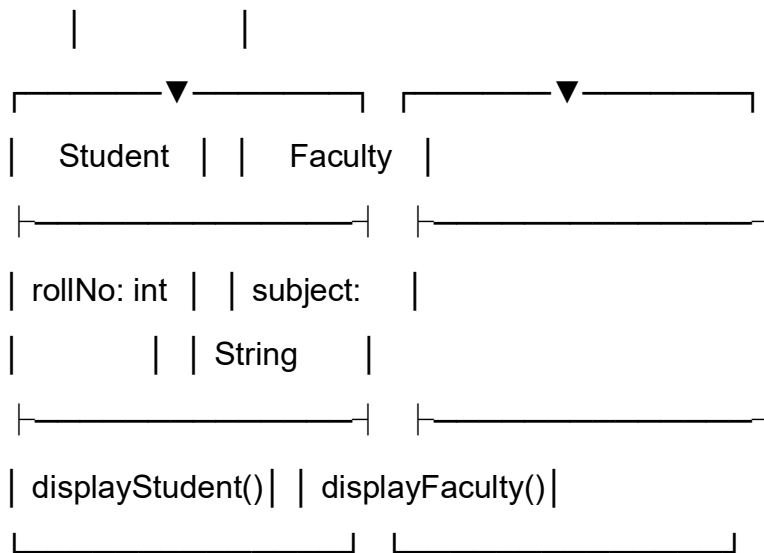
- Operating System: Windows / Linux / macOS
- Java Development Kit (JDK 17 or JDK 21)
- Text Editor or IDE such as Eclipse, IntelliJ IDEA, NetBeans, or VS Code
- Command Prompt / Terminal

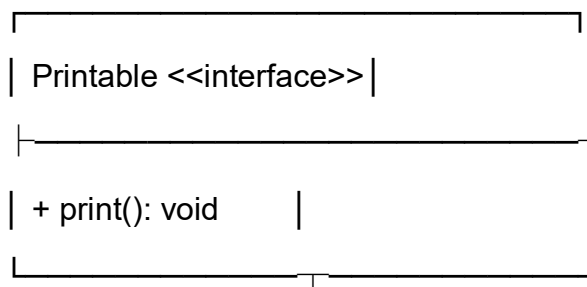
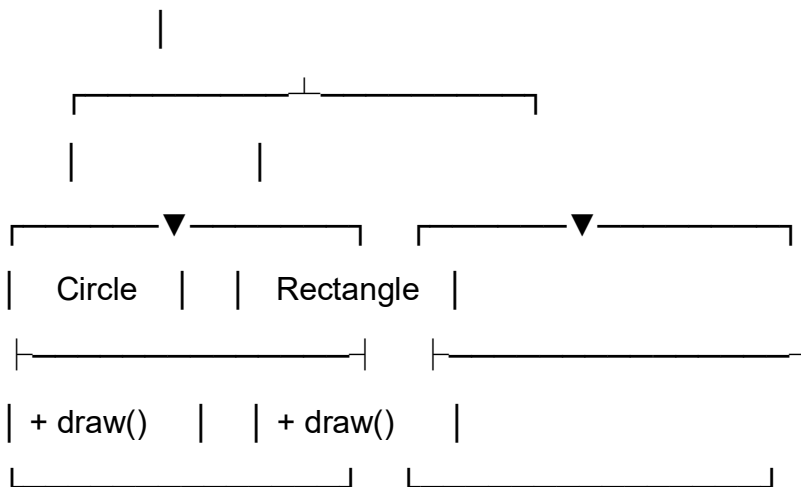
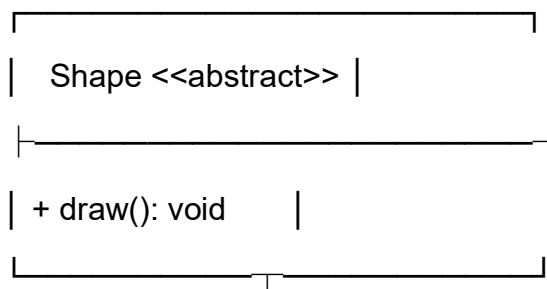
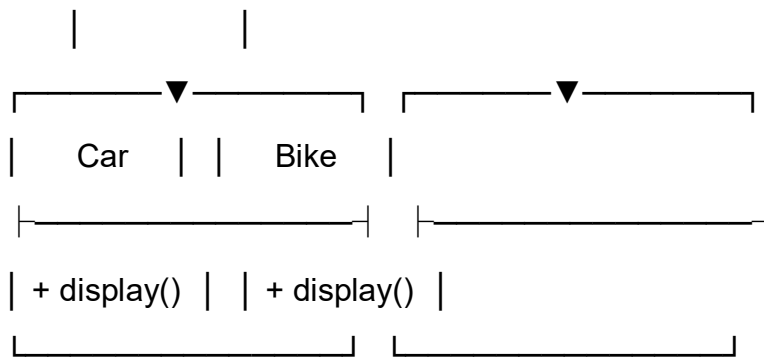
Algorithm / Logic Description

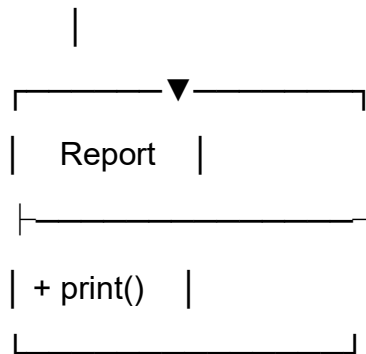
1. Create a "Book" class to demonstrate encapsulation and constructors.
2. Declare book details such as Book ID, Book Name, Author, and Price as private data members.
3. Create a default constructor and a parameterized constructor in the "Book" class.
4. Define setter and getter methods to access and modify the private book details.
5. Create a "Person" parent class containing common details such as name and age.
6. Create "Student" and "Faculty" classes that inherit from the "Person" class.
7. Define methods to display Student and Faculty details.
8. Create an "Area" class to demonstrate method overloading.
9. Define multiple "area()" methods for calculating the area of a square, rectangle, and circle.
10. Create a "Vehicle" parent class with a "display()" method.
11. Create "Car" and "Bike" classes that inherit from "Vehicle".
12. Override the "display()" method in the "Car" and "Bike" classes to demonstrate method overriding.
13. Create an abstract class "Shape" with an abstract "draw()" method.
14. Create "Circle" and "Rectangle" classes that inherit from "Shape" and implement the "draw()" method.
15. Create a "Printable" interface with a "print()" method.
16. Create a "Report" class that implements the "Printable" interface.
17. Create objects in the "main()" method to demonstrate all the above OOP concepts.
18. Display the results of each operation.

UML Class Diagram









Relationships:

- "Student" and "Faculty" inherit from "Person".
- "Car" and "Bike" inherit from "Vehicle".
- "Circle" and "Rectangle" inherit from the abstract "Shape" class.
- "Report" implements the "Printable" interface.
- "Area" demonstrates method overloading.
- "Book" demonstrates encapsulation and constructors.

Source Code

```
package javacore;
```

```
// Class Book demonstrates Encapsulation and Constructors
```

```
class Book {
```

```
    // Private data members
```

```
    private int bookId;
```

```
    private String bookName;
```

```
    private String author;
```

```
    private double price;
```

```
// Default Constructor
```

```
public Book() {  
    System.out.println("Default Constructor Called");  
}
```

```
// Parameterized Constructor
```

```
public Book(int id, String name, String author, double price) {  
    this.bookId = id;  
    this.bookName = name;  
    this.author = author;  
    this.price = price;  
}
```

```
// Setter method for Book ID
```

```
public void setBookId(int id) {  
    bookId = id;  
}
```

```
// Setter method for Book Name
```

```
public void setBookName(String name) {  
    bookName = name;  
}
```

```
// Setter method for Author
```

```
public void setAuthor(String a) {  
    author = a;  
}
```

```
// Setter method for Price
```

```
public void setPrice(double p) {  
    price = p;  
}
```

```
// Getter method for Book ID
```

```
public int getBookId() {  
    return bookId;  
}
```

```
// Getter method for Book Name
```

```
public String getBookName() {  
    return bookName;  
}
```

```
// Getter method for Author
```

```
public String getAuthor() {  
    return author;  
}
```

```
// Getter method for Price
```

```
public double getPrice() {  
    return price;  
}
```

```
// Method to display book details
```

```
public void display() {  
    System.out.println("Book ID : " + bookId);  
    System.out.println("Book Name : " + bookName);  
    System.out.println("Author : " + author);  
}
```

```
        System.out.println("Price : " + price);
    }
}
```

```
// Person class - Parent/Base class
```

```
class Person {
```

```
    // Data members
```

```
    String name = "sara";
```

```
    int age = 12;
```

```
}
```

```
// Student class inherits Person
```

```
class Student extends Person {
```

```
    // Student-specific data member
```

```
    int rollNo = 125;
```

```
    // Method to display student details
```

```
    void displayStudent() {
```

```
        System.out.println("\nStudent Details");
```

```
        System.out.println("Name : " + name);
```

```
        System.out.println("Age : " + age);
```

```
        System.out.println("Roll No : " + rollNo);
```

```
    }
```

```
}
```

```
// Faculty class inherits Person
class Faculty extends Person {

    // Faculty-specific data member
    String subject = "Java";

    // Method to display faculty details
    void displayFaculty() {
        System.out.println("\nFaculty Details");
        System.out.println("Name : " + name);
        System.out.println("Age : " + age);
        System.out.println("Subject : " + subject);
    }
}
```

```
// Area class demonstrates Method Overloading
class Area {
```

```
    // Calculate area of square
    void area(int side) {
        System.out.println("\nArea of Square = " + (side * side));
    }
```

```
    // Calculate area of rectangle
    void area(int length, int breadth) {
        System.out.println(
            "Area of Rectangle = " + (length * breadth)
```

```
    );  
}  
  
// Calculate area of circle  
void area(double radius) {  
    System.out.println(  
        "Area of Circle = " + (3.14 * radius * radius)  
    );  
}  
}
```

```
// Vehicle class - Parent class  
// Demonstrates Method Overriding  
class Vehicle {  
  
    // Parent class display method  
    void display() {  
        System.out.println("\nThis is a Vehicle");  
    }  
}
```

```
// Car class inherits Vehicle  
class Car extends Vehicle {  
  
    // Overriding display method  
    void display() {  
        System.out.println("This is a Car");  
    }  
}
```

```
}  
}
```

```
// Bike class inherits Vehicle  
class Bike extends Vehicle {
```

```
    // Overriding display method  
    void display() {  
        System.out.println("This is a Bike");  
    }  
}
```

```
// Abstract class Shape  
abstract class Shape {
```

```
    // Abstract method  
    abstract void draw();  
}
```

```
// Circle class inherits Shape  
class Circle extends Shape {
```

```
    // Implementing abstract method  
    void draw() {  
        System.out.println("\nDrawing Circle");  
    }  
}
```

```
}
```

```
// Rectangle class inherits Shape
```

```
class Rectangle extends Shape {
```

```
    // Implementing abstract method
```

```
    void draw() {
```

```
        System.out.println("Drawing Rectangle");
```

```
    }
```

```
}
```

```
// Interface Printable
```

```
interface Printable {
```

```
    // Abstract method of interface
```

```
    void print();
```

```
}
```

```
// Report class implements Printable interface
```

```
class Report implements Printable {
```

```
    // Implementing interface method
```

```
    public void print() {
```

```
        System.out.println("\nPrinting Library Report");
```

```
    }
```

```
}
```

```
// Main class of the program
public class LibraryManagementSystem {

    // Main method - program execution starts here
    public static void main(String[] args) {

        // Creating Book object using default constructor
        Book b1 = new Book();

        // Setting book details using setter methods
        b1.setBookId(101);
        b1.setBookName("Java Programming");
        b1.setAuthor("James Gosling");
        b1.setPrice(550);

        // Displaying first book details
        System.out.println("\nBook Details");
        b1.display();

        // Creating Book object using parameterized constructor
        Book b2 =
            new Book(102, "Python", "Guido", 700);

        // Displaying second book details
        System.out.println("\nParameterized Constructor");
        b2.display();
```

```
// Creating Student object
// Demonstrates Inheritance
Student s = new Student();
```

```
// Displaying student details
s.displayStudent();
```

```
// Creating Faculty object
// Demonstrates Inheritance
Faculty f = new Faculty();
```

```
// Displaying faculty details
f.displayFaculty();
```

```
// ----- Method Overloading -----
```

```
// Creating Area object
Area a = new Area();
```

```
// Calling overloaded method for Square
a.area(5);
```

```
// Calling overloaded method for Rectangle
a.area(10, 20);
```

```
// Calling overloaded method for Circle
```

```
a.area(4.5);
```

```
// ----- Method Overriding -----
```

```
// Creating Vehicle reference
```

```
Vehicle v;
```

```
// Vehicle reference refers to Car object
```

```
v = new Car();
```

```
// Calls Car's overridden display() method
```

```
v.display();
```

```
// Vehicle reference refers to Bike object
```

```
v = new Bike();
```

```
// Calls Bike's overridden display() method
```

```
v.display();
```

```
// ----- Abstraction -----
```

```
// Creating Shape reference
```

```
Shape sh;
```

```
// Shape reference refers to Circle object
```

```
sh = new Circle();
```

```

// Calls Circle's draw() method
sh.draw();

// Shape reference refers to Rectangle object
sh = new Rectangle();

// Calls Rectangle's draw() method
sh.draw();

// ----- Interface -----

// Creating Report object
Report r = new Report();

// Calling method implemented from Printable interface
r.print();
}
}

---
```

Compilation & Execution Commands

Save the source code as:

LibraryManagementSystem.java

If you are using the "package javacore;" statement, save the file inside the appropriate "javacore" package/folder.

Compile the program:

```
javac LibraryManagementSystem.java
```

Run the program:

```
java LibraryManagementSystem
```

Output Screen

Default Constructor Called

Book Details

Book ID : 101

Book Name : Java Programming

Author : James Gosling

Price : 550.0

Parameterized Constructor

Book ID : 102

Book Name : Python

Author : Guido

Price : 700.0

Student Details

Name : sara

Age : 12

Roll No : 125

Faculty Details

Name : sara

Age : 12

Subject : Java

Area of Square = 25

Area of Rectangle = 200

Area of Circle = 63.585

This is a Car

This is a Bike

Drawing Circle

Drawing Rectangle

Printing Library Report

Result

The Library Management System was successfully implemented in Java. The program demonstrates important Object-Oriented Programming concepts, including encapsulation, default and parameterized constructors, inheritance, method overloading, method overriding, abstraction, and interfaces.